

```

# =====
# Name: Srushti Kale
# PRN: 23070521067
# Practical 10: Credit Card Fraud Detection using Deep Learning
# =====

import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE

# 1. Synthesize Highly Imbalanced Credit Card Transaction Data
# (Mimics real-world setups where fraud cases are ~0.1 - 1% of total)
X, y = make_classification(
    n_samples=20000,
    n_features=29,
    n_informative=24,
    n_redundant=5,
    weights=[0.99, 0.01],
    random_state=42
)

print(f"Original class counts: Non-Fraud (0) = {np.sum(y == 0)}, Fraud (1) = {np.sum(y == 1)}")

# 2. Train-Test Split & Preprocessing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 3. Handle Imbalanced Data using SMOTE (Synthetic Minority Over-sampling Technique)
print("\nApplying SMOTE to balance the training set...")
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

print(f"Resampled training counts: Non-Fraud = {np.sum(y_train_resampled == 0)}, Fraud = {np.sum(y_train_resampled == 1)}")

# 4. Build Deep Neural Network Classifier
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

# 5. Compile Model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=[
        tf.keras.metrics.Precision(name='precision'),
        tf.keras.metrics.Recall(name='recall')
    ]
)

```

```
model.summary()

# 6. Train the Model
print("\n--- Training Deep Neural Network ---")
history = model.fit(
    X_train_resampled, y_train_resampled,
    validation_split=0.2,
    epochs=15,
    batch_size=64,
    verbose=1
)

# 7. Evaluate Precision, Recall, and F1-Score
print("\n--- Evaluating Model Performance ---")
y_pred_probs = model.predict(X_test)
y_pred = (y_pred_probs > 0.5).astype(int).flatten()

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report (Precision, Recall, F1-Score):")
print(classification_report(y_test, y_pred, target_names=["Normal Transaction", "Fraudulent"]))
```

Original class counts: Non-Fraud (0) = 19709, Fraud (1) = 291

Applying SMOTE to balance the training set...

Resampled training counts: Non-Fraud = 15767, Fraud = 15767

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,920
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 4,545 (17.75 KB)

Trainable params: 4,545 (17.75 KB)

Non-trainable params: 0 (0.00 B)

--- Training Deep Neural Network ---

Epoch 1/15

395/395 10s 11ms/step - loss: 0.4863 - precision: 0.7918 - recall: 0.5203 - \

Epoch 2/15

395/395 2s 4ms/step - loss: 0.3279 - precision: 0.8735 - recall: 0.7315 - \

Epoch 3/15

395/395 2s 4ms/step - loss: 0.2688 - precision: 0.8941 - recall: 0.7930 - \

Epoch 4/15

395/395 2s 5ms/step - loss: 0.2264 - precision: 0.9071 - recall: 0.8376 - \

Epoch 5/15

395/395 2s 6ms/step - loss: 0.1983 - precision: 0.9114 - recall: 0.8662 - \

Epoch 6/15

395/395 2s 5ms/step - loss: 0.1739 - precision: 0.9205 - recall: 0.8899 - \

Epoch 7/15

395/395 2s 5ms/step - loss: 0.1572 - precision: 0.9239 - recall: 0.9068 - \

Epoch 8/15

395/395 1s 4ms/step - loss: 0.1446 - precision: 0.9306 - recall: 0.9131 - \

Epoch 9/15

395/395 2s 4ms/step - loss: 0.1390 - precision: 0.9321 - recall: 0.9240 - \

Epoch 10/15

395/395 2s 4ms/step - loss: 0.1224 - precision: 0.9378 - recall: 0.9333 - \

Epoch 11/15

395/395 2s 5ms/step - loss: 0.1209 - precision: 0.9384 - recall: 0.9353 - \

Epoch 12/15

395/395 3s 5ms/step - loss: 0.1175 - precision: 0.9386 - recall: 0.9396 - \

Epoch 13/15

395/395 2s 4ms/step - loss: 0.1075 - precision: 0.9445 - recall: 0.9470 - \

Epoch 14/15

395/395 2s 4ms/step - loss: 0.1073 - precision: 0.9444 - recall: 0.9468 - \

Epoch 15/15

395/395 1s 4ms/step - loss: 0.1054 - precision: 0.9418 - recall: 0.9493 - \